Routing Algorithms, Autonomous Systems, and BGP

Lecture 24

Computer Networks and Distributed Programming

Where the rows of the table come from

- For four lectures the forwarding table has been given: a row is a prefix, the row names an outgoing interface, the longest match wins.
- Nobody has said who writes the rows. That is the control plane, and we have already named the two ways it can be organised.
- Either an algorithm runs inside every router and the copies talk to each other, or one controller computes all the tables and hands them out.
- Today we open the box: what such an algorithm computes, and how routers that can see only their own neighbourhood end up agreeing on end-to-end paths.
- Two families of algorithm, both in use — and then the reason why neither of them runs over the Internet as a whole.

What a routing algorithm must produce

- The output is a path: a sequence of routers from sender to receiver, for every destination and from every source.
- A good path is usually one of least cost — and cost may mean the length of the link, its speed, or what it costs in money.
- In the real world policy comes in too: a rule such as *router x, owned by organisation Y, must not carry packets coming from Z*. It has nothing to do with distance.
- The problem is not only ours. Finding good paths on a weighted graph is the same problem in games, in robotics, in driving directions.
- What is ours is the constraint: no node has the map, and the map changes while you are computing on it.

Flooding

- The simplest answer that works at all: forward every incoming packet on every link except the one it arrived on.
- No table, no computation, no agreement between routers. If a path exists the packet will arrive, because every path is tried, the best one included.
- The copies that go nowhere die when their hop limit runs out — the same field that makes a trace expire one router further along each time.
- It is extremely robust: routers may be down, links may be cut, and the redundancy covers it. That is why it survives where redundancy is the requirement.
- And it is hopeless on a large fast network: every packet becomes many, and the many become more. For the Internet we need something that decides one outgoing link.

The network as a graph

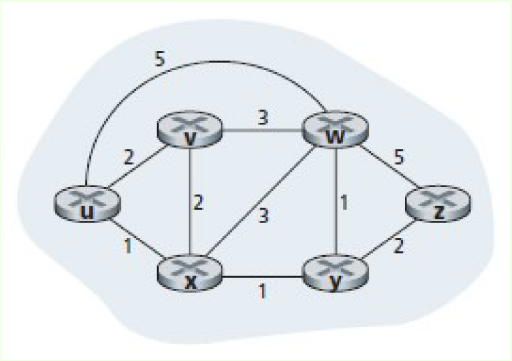
Definition

A network is a graph $G = (N, E)$: N the set of nodes, the routers; $E \subseteq N \times N$ a binary relation, the edges, the physical links between them.

- The relation is symmetric: $(x, y) \in E \Rightarrow (y, x) \in E$. The graph is undirected, and y is a neighbour of x when $(x, y) \in E$.
- Every edge carries a cost $c(x, y)$, with $c(x, y) = c(y, x)$; and $c(x, y) = \infty$ when $(x, y) \notin E$, so the function is defined on all pairs and non-existence is just an infinite price.
- Where costs come from is somebody else's problem: for us they are given. The algorithms extend to directed links with a different cost each way.

The cost of a path

- A path is a sequence $\pi = (x_1, \ldots, x_p)$ in which every adjacent pair is an edge.
- Its cost is the sum along it: $c(\pi) = \sum_{i=1}^{p-1} c(x_i, x_{i+1})$.
- Between two nodes there are typically many paths — in this graph seventeen from u to z — and one or more of them have least cost.
- If every edge costs the same, the least-cost path is also the shortest: the one with fewest links.



Which is the least-cost path from u to w? And how did you find it?

Least-cost algorithms: two families

- We do not want the best path between two nodes but between all pairs, and an algorithm has converged when every node holds its own least-cost paths.
- The families differ in what a node knows, not in what it computes.
- Global: the algorithm is given the whole topology and all the link costs before it starts. Since it needs the cost of every link, these are the link-state algorithms.
- Decentralised: a node starts knowing only the cost of its own links, and gets everything else by talking to its neighbours, over and over. These are the distance-vector algorithms.
- Careful: global does not mean *computed in one place*: a link-state algorithm normally runs in every router, each with the same map.

Distance vector: nobody sees the whole map

- Each node holds a vector of estimates: what it believes the least cost to be, towards every destination in the network.
- Distributed: a node receives estimates from its directly attached neighbours, computes, and sends the result of the computation back to them.
- Iterative: the exchange continues until there is nothing new to say — and it is self-terminating: no signal announces the end, it simply stops.
- Asynchronous: the nodes are not in lockstep. Each acts when news reaches it, which is what makes the thing usable on a real network where routers reboot.
- Underneath it there is one equation, and it is worth stating on its own.

The Bellman-Ford equation

Definition

Let $d_x(y)$ be the cost of the least-cost path from x to y. Then

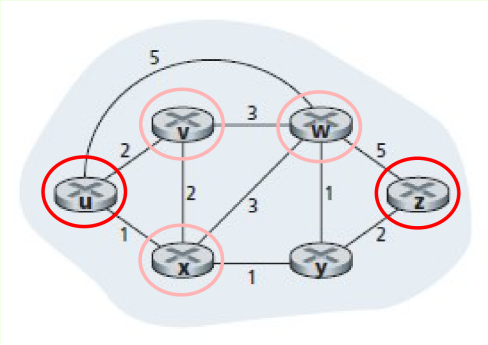
$$d_x(y) \;=\; \min_v \{\, c(x,v) + d_v(y) \,\}$$

where v ranges over the neighbours of x.

- The reason is almost a tautology: to reach y, x must first step to some neighbour v, and from there the best it can do is $d_v(y)$.
- So the minimum is taken over a handful of sums, one per neighbour: $c(x, v_1) + d_{v_1}(y)$, $c(x, v_2) + d_{v_2}(y)$, and so on.
- Nothing in it mentions the topology beyond x. That is the whole point, and later it will also be the whole problem.

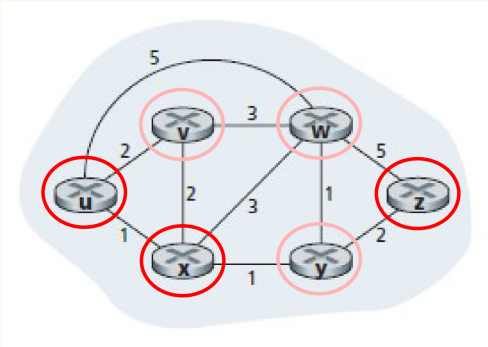
Checking it at u

- Take $x = \mathrm{u}$ and $y = \mathrm{z}$. Node u has three neighbours: v, w and x.
- Walking the graph by hand gives $d_v(\mathrm{z}) = 5$, $d_w(\mathrm{z}) = 3$, $d_x(\mathrm{z}) = 3$; the first hops cost 2, 5 and 1.
- So $d_u(\mathrm{z}) = \min\{2{+}5,\ 5{+}3,\ 1{+}3\} = 4$, which is what the picture says.
- The minimum is achieved by x, and x is indeed the second node of the least-cost path u, x, y, z.



The winning neighbour is the table entry

- The same equation holds at every node of that path: repeat it at x and y comes out, repeat it at y and z comes out.
- And the useful part is not the number 4. It is the neighbour v^* that achieves the minimum.
- Node u writes in its forwarding table: for destination z, use the link towards x. It never stores the path, only the first hop.
- To evaluate the equation a node needs an estimate $D_v(y)$ for each neighbour v — and that estimate is exactly what the neighbours keep sending it.



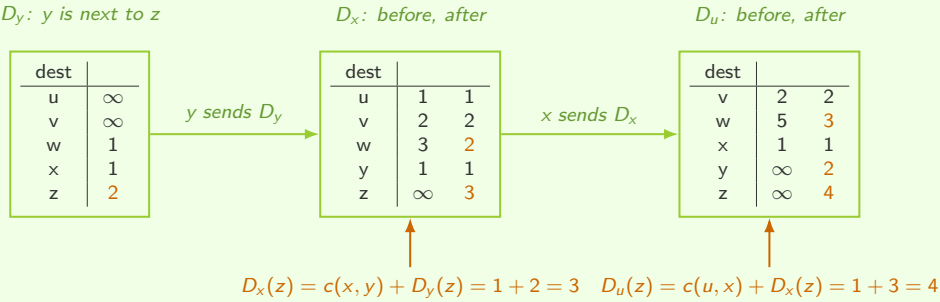
The algorithm

```
1  DistanceVector(x):
2    # initialization: only the direct links are known
3    for each destination y in N:
4        D_x(y) = c(x,y)            # ∞ if y is not a neighbour
5    for each neighbour w, for each y in N:
6        D_w(y) = unknown
7    send D_x(·) to every neighbour
8
9    loop forever:
10       wait until a link cost to a neighbour changes
11            or a distance vector arrives from a neighbour
12       for each y in N:
13           D_x(y) = min_v{ c(x,v) + D_v(y) }
14       if D_x(·) changed:
15           send D_x(·) to every neighbour
```

Nothing here is periodic: a node waits, and speaks only when one of its own numbers has moved. When nobody speaks, the network is converged.

How the news travels

- At the start u cannot reach z at all — $D_u(z) = \infty$ — and neither can x. Only y knows anything: its own link to z costs 2.



nobody computed the path u, x, y, z: each node added one link cost to a number it was told — and u dropped its own direct link to w, which costs 5, for a path through x that costs 3

What it costs, and what it costs you

- The traffic the algorithm generates is small and local: a node speaks only to its neighbours, and only when one of its least-cost paths has actually changed.
- There is no fixed number of rounds. How long convergence takes depends on the topology and on where the change happened: news has to walk, one hop per exchange.
- In favour: it is asynchronous. A router may come up at any moment, ask nobody's permission, and be absorbed by its neighbours.
- Against: good news travels fast and bad news travels slowly. A cheaper path is adopted within a couple of exchanges; a link that gets worse takes far longer, and meanwhile there may be loops.
- That second one has a name and deserves the next three frames.

Why the numbers can be wrong

Can I be sure I am not on my neighbour's best path?

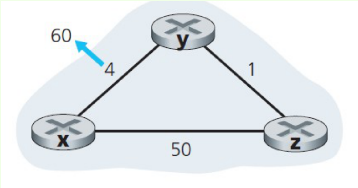
A node computes $D_x(y)$ out of $D_v(y)$, the number its neighbour sent. That number is a distance and not a path: nothing in it says which nodes it goes through. If it goes through x, then x is adding a link cost to a value that already contains that link.

- This is not a bug in the pseudocode. It is the locality that makes distance vector cheap: nobody carries paths around, only totals.
- While costs are falling nothing goes wrong — a smaller number is a smaller number. The trouble starts when a cost rises.

Bad news travels slowly

A link gets dearer, and only one of its ends notices

$c(x, y)$ jumps from 4 to 60, and only y sees it. y computes $D_y(x) = \min\{60{+}0,\ 1{+}5\} = 6$ and routes to x through z — while z routes to x through y.



- At that instant there is a routing loop: a packet for x bounces between y and z.
- z hears 6 and computes $1{+}6 = 7$; y computes 8; z computes 9. The two of them count upwards.
  - It does stop — after 44 exchanges, when z's path through y exceeds its direct link of 50. With 10 000 and 9 999, much later: hence the name.

Poisoned reverse

- His first remark: if x had noticed the change too and said so, the two would have settled at once. Both ends of a link normally do see it; the delay is the whole problem.
- The standard defence is a lie. Poisoned reverse: if z reaches x through y, then z tells y that its own distance to x is ∞ — even though z knows perfectly well it is 5.
- As long as y believes z has no path to x, y will never try to route to x through z, which is exactly the move that started the count.
- So when the link goes to 60, y keeps the direct link at 60 and says so; z switches to its own link at 50 and now tells the truth; y settles at 51. Three exchanges instead of forty-four.
- And it is not a cure: a loop of three or more nodes is not detected this way. The general problem stays.

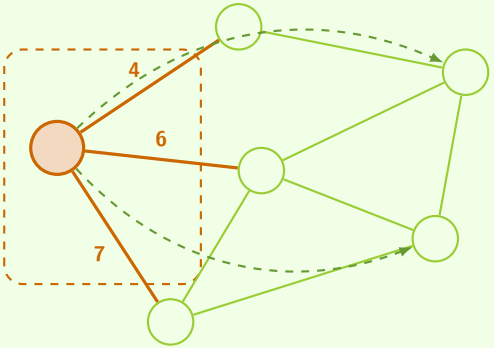
Link state: everybody holds the same map

- The opposite hypothesis: every node knows the whole graph — all the nodes, all the links, all the costs — before it starts computing.
- Then no estimate is built out of somebody else's estimate, and there is nothing to count to infinity: each node works on facts, not on hearsay.
- The algorithm is Dijkstra's, and it computes the least-cost paths from one source to every destination. So it has to be run at every node, each with itself as the source.
- Whether the map is assembled inside each router or inside a single controller changes nothing about the computation — the controller version is precisely the centralised control plane we have already drawn.
- What is left to explain is how a node obtains a map of a network it cannot see.

Building the map

- Discover: send a `hello` on every link; whoever is at the far end answers with its identity. Measure: set the cost towards each neighbour, measured or configured.
- Announce: build one packet listing your neighbours and the cost of reaching them, and broadcast it — to every router, not only to the adjacent ones. Collect: receive everybody else's.

what it knows first-hand: the cost of its own links



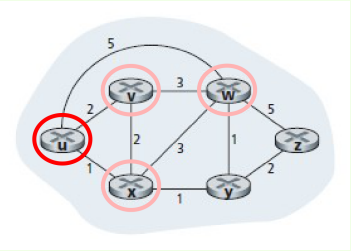
and it goes to every node, not only to the neighbours — so afterwards everybody holds the same map

Dijkstra

```
1  LinkState(u):
2    # initialization: the whole graph is already known
3    N' = {u}
4    for each node v:
5        if v is a neighbour of u:  D(v) = c(u,v);   p(v) = u
6        else:                      D(v) = ∞
7
8    repeat:
9        find w ∉ N' with D(w) minimum
10       add w to N'
11       for each neighbour v of w not in N':
12           if D(w) + c(w,v) < D(v):
13               D(v) = D(w) + c(w,v);   p(v) = w
14   until N' = N
```

$D(v)$: best cost to v known so far. $p(v)$: the node just before v on it. N': the nodes whose least cost is settled for good.

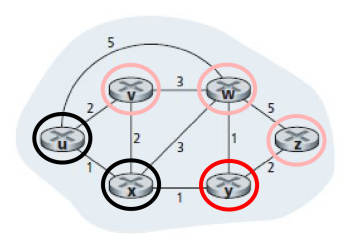
Running it from u: the start



step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1,u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

- The image holds the whole run: read the boxed row, the rest is where we are going.
- $N' = \{u\}$, and the three neighbours get their direct links: v at 2, x at 1, w at 5. Everything else is ∞.
- w is at 5 although a cheaper way to it exists: 5 is the direct link, and it is all u knows so far.

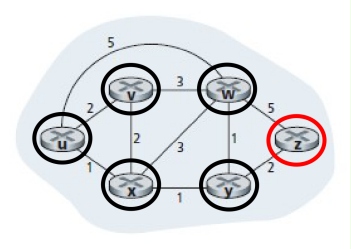
Two iterations, and the direct link to w loses twice



step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1,u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

- Step 1: the smallest outstanding value is x, at 1: x enters N', and its neighbours are relaxed — w falls from 5 to 4 through x, and y becomes 2.
- Step 2: v and y are both at 2. The tie is broken arbitrarily and y enters; w falls again, to 3 through y, and z becomes 4.
- Two relaxations, and the one-hop link of cost 5 has been beaten twice by a three-hop path.

The end of the run

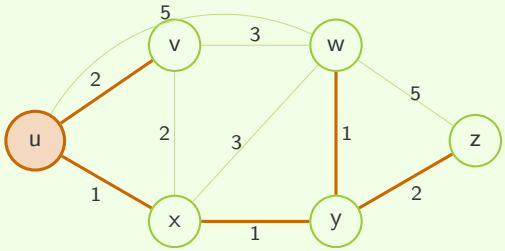


step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1,u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

- Steps 3, 4 and 5 admit v, w and z and change nothing: every value they could improve is already final.
- After k iterations the k nearest destinations are settled, so once a node enters N' its value never moves again.
- One iteration per node, and the last row is empty because there is nothing left to relax.

From predecessors to a forwarding table

- The table has computed $p(\cdot)$ as well: z came from y, y from x, x from u. Backtracking it gives the path, and doing it for every destination gives a tree rooted at u.



backtracking $p(\cdot)$ from every destination gives one tree rooted at u, not five separate paths

destination	link
v	(u, v)
w	(u, x)
x	(u, x)
y	(u, x)
z	(u, x)

of the whole tree the table keeps the first link only: this is the row we took as given

What it costs

- Each iteration searches the nodes not yet in N' for the smallest value: n of them, then $n-1$, then $n-2$. Summed, $n(n+1)/2$ comparisons, so $O(|N|^2)$.
- With a better data structure — a heap — finding that minimum costs logarithmic rather than linear time, and the bound improves accordingly.
- The messages are the other half of the price: every node's link state must reach every node, which is $O(|N|\,|E|)$ messages, and again on every change.
- In favour: convergence is fast and there are no counting loops: the state of every link is propagated as a fact.
- Against: a node can compute nothing until the map has arrived. It must first hear from the whole network.

When the cost of a link is its own traffic

What if we set the cost of a link to the load it carries?

Then the algorithm changes the costs it has just read. Everyone computes that the clockwise way is cheaper and moves there, which makes it expensive and the other side empty; at the next run everyone moves back. The traffic swings and never settles.

- It is not a defect of link state: any algorithm with a load-based metric oscillates the same way.
- Making the cost independent of the load looks unacceptable — avoiding congested links is the whole point — and yet it is what is done: RIP, OSPF and BGP are all load-insensitive.
- A partial remedy for what remains: routers must not run the algorithm at the same instant. Left alone they synchronise themselves, so the time of each advertisement is randomised.

Distance vector against link state

	Distance vector	Link state
A node knows	its links, plus neighbours' vectors	the whole graph
It talks to	its neighbours only	every node, by broadcast
It says	its distance to everybody	the cost of its own links
Messages	only when a best path changes	$O(\|N\|\,\|E\|)$, and on every change
Convergence	slow, with loops meanwhile	$O(\|N\|^2)$ once the map is in

- Robustness: under LS a node computes its own table, so a wrong advertisement is confined to that node's links. Under DV every calculation feeds the next one, and a wrong value diffuses.
- In 1997 a malfunctioning router at a small ISP fed the backbone wrong routes; the backbone flooded it with traffic and parts of the Internet were unreachable for hours.
- Neither wins. Both are used in the Internet — and the reason is what comes next.

Two things the graph model has ignored

- Scale: the Internet holds hundreds of millions of routers. One row per destination is impossible memory, broadcasting every link change is impossible traffic, and a distance vector iterating over that many nodes would never converge.
- Autonomy: an ISP wants to run whatever algorithm it likes inside its own network, and to keep the shape of that network to itself. A single worldwide algorithm asks it to publish everything.
- One move solves both: group the routers into autonomous systems, each under a single administrative control. Often one ISP is one AS; some large ones are several.
- An AS is identified by a globally unique number, its ASN, handed out by the same regional registries that hand out addresses.
- Inside an AS everybody runs the same algorithm and knows about everybody else — an intra-AS protocol. Between ASs it has to be something else.

OSPF: link state inside one AS

- OSPF is a link-state protocol: flooding of link-state advertisements, then Dijkstra run locally with the router itself as the root. The *Open* means the specification is public.
- The weights are the administrator's business, not the protocol's. All ones gives minimum-hop routing; weights inversely proportional to capacity push traffic off the slow links.
- A `HELLO` on each link checks the neighbour is alive. A new advertisement goes out whenever a link changes state, and at least every 30 minutes even when nothing has changed.
- Advertisements are broadcast to every router in the AS, not to the neighbours: it is the map of the whole AS that has to be rebuilt.
- Note where it sits: OSPF messages are carried directly by IP, protocol number 89. Not TCP, not UDP — so it has to provide its own reliable delivery.

Areas, and the rest of OSPF

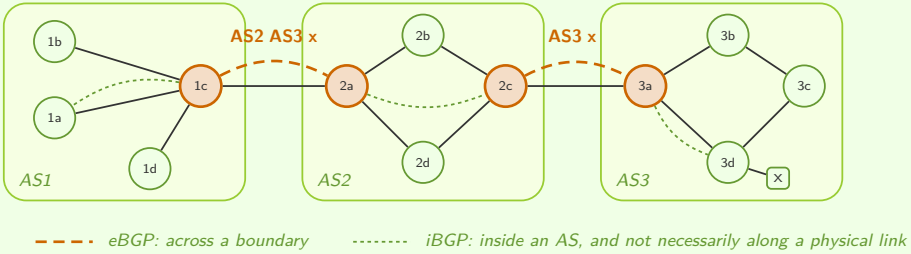
- Even one AS can be too large for a single map. It can be cut into areas: each area runs its own link-state algorithm, and a router floods its advertisements only inside its area.
- Exactly one area is the backbone, and every area border router belongs to it. Crossing the AS means: up to a border router, across the backbone, down into the destination area.
- It is the same move as cutting subnets inside an address block, one level further up: hide the detail, keep the summary.
- When several paths to a destination have the same cost, OSPF may use all of them rather than picking one.
- Exchanges can be authenticated, by shared password or by a keyed hash (MD5 once, HMAC-SHA today). By default they are not, and an unauthenticated advertisement can be forged — which is one way to empty a network.

BGP: the protocol that has to be the same for everybody

- An inter-AS protocol only works if the ASs on both sides run the same one, so there is exactly one: BGP, and every AS in the Internet speaks it.
- Its destinations are not addresses but CIDR prefixes: a route to `138.16.68/22` covers 1024 addresses in one row. This is where those aggregated blocks pay off.
- First job: learn from neighbouring ASs which prefixes exist and through which sequence of ASs they can be reached.
- Second job: choose, among the routes learned for a prefix, the one to install — and the criterion will not be distance.
- Without it a subnet is an island: it exists, it has addresses, and no router anywhere has a row that matches them.

eBGP and iBGP

- Autonomous systems do not talk: routers do. Two of them keep a semi-permanent TCP connection on port 179 — a routing protocol built on the transport service we have already used.
- Across a boundary it is eBGP; between routers of the same AS it is iBGP, and the notice of a foreign prefix travels inwards on it.



What a route carries

Route

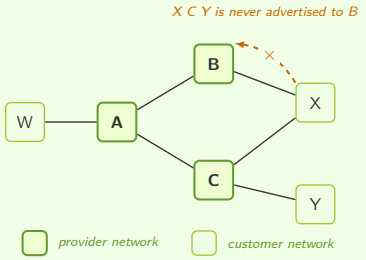
A prefix together with its attributes. Two of them matter here.

- AS-PATH: the list of ASs the advertisement has crossed. Each AS prepends its own number before passing it on, so the list grows as the news travels.
- It also detects loops: a router that finds its own AS already in the list rejects the advertisement. No hop count, no counting to infinity.
- NEXT-HOP: the IP address of the router interface that begins the AS-PATH — an address that does not belong to your AS, but sits on a subnet directly attached to it.
- NEXT-HOP is the joint between the two protocols: to use a BGP route, a router looks up that address in the table its intra-AS protocol built.

Choosing among routes

- A router often learns dozens of routes to the same prefix. The rules are applied in order, each eliminating some, until one survives.
- 1. Local preference: a value attached to the route by the administrator. It is a policy number, decided inside the AS, and it comes first — before anything about distance.
- 2. Shortest AS-PATH: fewest autonomous systems crossed. Not fewest routers: between ASs there is no other notion of cost.
- 3. Hot potato: of the survivors, the one whose NEXT-HOP is nearest by the intra-AS cost. Get the packet out of my own AS as cheaply as I can, and never mind the rest of its journey.
- Rule 2 before rule 3 is what keeps BGP from being purely selfish. And two routers of the same AS may still pick different AS paths for the same prefix.

Policy decides before cost does



X knows the path through C to Y, and keeps quiet about it: B never learned it, so B never sends X any traffic for Y — no rule forbids the transit, the silence does

- Should B pass on to C the path it learned to W? Carrying transit between A and C costs B money and earns it nothing — and this is the money side of the question we left open with net neutrality.
- The rule of thumb, in no standard: traffic on a provider's backbone must have a source or a destination in one of its customers. The rest is bilateral and confidential.

Why two different protocols at all

- Policy: between ASs it dominates: who may carry whose traffic is a commercial question, and BGP carries the attributes that let it be answered. Inside one AS everything is under one administration, and the question does not arise.
- Scale: between ASs the destinations are the whole Internet. Inside, if a network grows too large, it can always be split into two ASs — and OSPF can split further into areas.
- Performance: between ASs the quality of the route is secondary: a longer path that satisfies a policy beats a shorter one that does not, and there is no cost beyond AS hops. Inside an AS, routing can afford to optimise.
- Which is why the answer to *which algorithm does the Internet use* is not one algorithm but a division of labour.

One address, many machines

- IP anycast: give the same IP address to many machines in different parts of the world, and have each of them advertise it with BGP.
- A router that receives several advertisements for that address believes they are different paths to the same place, and runs the selection algorithm. It picks the nearest copy without knowing there are copies.
- This is what was behind a fact we left unexplained: the DNS root has thirteen addresses and more than a thousand machines. The query goes to the nearest one holding that address.
- And the limit: content networks mostly do not use it: a BGP change in the middle of a transfer can deliver later segments of the same TCP connection to a different machine, which has never heard of it.

Putting a company on the Internet

- Contract with a local ISP: a physical connection from your gateway router to one of theirs, and a block of addresses, say a `/24`. Assign them to the web server, the mail server, the DNS server, the gateway.
- Contract with a registrar for a domain name, and give it the address of your DNS server: the registrar puts that entry in the `.com` servers, so anyone can find you by name.
- In your own DNS server, map `www.your-company.com` to the address of your web server.
- And the step without which none of it works: your ISP advertises your prefix with BGP. The advertisement propagates outwards, and eventually every router in the Internet holds a row that matches your addresses — or an aggregate that contains them.
- Addressing, naming, forwarding and routing, in one procedure.

Summary

- Routing computes what forwarding consumes. On a graph (N, E) with costs, the object wanted is the least-cost path — and what gets stored is only its first link.
- Distance vector: Bellman-Ford, talk to your neighbours only, send them your totals. Cheap and asynchronous, slow to converge, and it counts to infinity when a cost rises; poisoned reverse helps and does not cure.
- Link state: broadcast the cost of your own links, receive everybody's, run Dijkstra on the map. No loops, $O(|N|^2)$, and nothing at all until the map has arrived.
- Neither metric follows the traffic, because a cost that follows the traffic makes the routing oscillate.
- Scale and autonomy cut the Internet into autonomous systems: OSPF inside, BGP between. BGP routes to prefixes, and decides by policy first and cost afterwards.
- Every path we computed is a sequence of single hops, and we have never asked what one hop actually is: how the bits are framed on one link, and how the machine at the other end is named there. That is where we go next.